MBS-fast Manual

August 12, 2026

Contents

MBS-fast computes light scattering by non-spherical particles with geometrical ray tracing plus Physical Optics (PO) diffraction. The code is optimized for CPU OpenMP/MPI runs and CUDA GPU diffraction runs. This manual describes the current command line interface, build targets, grids, multi-GPU scans, and the main diffraction/Jones/Mueller formulas used by the implementation.

1 Build

1.1 Requirements

Component	Required for	Notes
GCC >= 9 or Clang >= 14	CPU and host CUDA code	GCC is the normal path on EPYC
OpenMP	CPU parallelism	GCC links with `libgomp`
MPI	`cpu/` split build	`mpicxx` is used when available
CUDA toolkit	`gpu/` split build	`nvcc`, `libcudart`, `libcufft`
NVIDIA driver	GPU runs	Must support the requested `sm_XX` binary

The split build is the recommended build path. CPU objects are kept under `cpu/build/`; CUDA objects are kept under `gpu/build/`. The shared physics code remains in `src/`.

1.2 CPU build

```
# MPI + OpenMP CPU binary
make -C cpu -j

# Run one MPI rank with 64 OpenMP threads
OMP_NUM_THREADS=64 cpu/bin/mbs_po_mpi --po -p 1 125.9 78.09 \
    --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
    --grid 0 180 600 180 --threads 64 --close -o out_cpu

# Run four MPI ranks, each with 16 OpenMP threads
mpirun -np 4 cpu/bin/mbs_po_mpi --po -p 1 125.9 78.09 \
    --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
    --grid 0 180 600 180 --threads 16 --close -o out_cpu_mpi
```

Debug help build:

```
make -C cpu debug
cpu/bin/mbs_po_mpi_debug --help-debug
```

1.3 GPU build

```
# Default GPU binary: float + fast math
PATH=/usr/local/cuda/bin:$PATH make -C gpu -j

# Explicit variants
make -C gpu float       -j   # gpu/bin/mbs_po_gpu_float
make -C gpu float_fast  -j   # gpu/bin/mbs_po_gpu_float_fast
make -C gpu double_fast -j   # gpu/bin/mbs_po_gpu_double_fast
```

Target	Binary	CUDA precision	Fast math	Typical use
`make -C gpu`	gpu/bin/mbs_po_gpu_float_fast	FP32	yes	Fast production scans
`make -C gpu float`	gpu/bin/mbs_po_gpu_float	FP32	no	FP32 check without fast math
`make -C gpu double_fast`	gpu/bin/mbs_po_gpu_double_fast	FP64	yes	Reference GPU checks

Target	Binary	CUDA precision	Fast math	Typical use
`make -C gpu double_debug`	gpu/bin/mbs_po_gpu_double_debug	FP64	yes	Debug help and diagnostics

The GPU split build enables CUDA diffraction by default. `--gpu` is accepted but optional. Use `--cpu` only when you deliberately want the CPU diffraction backend from a GPU-capable binary.

The Makefile detects the GPU compute capability with `nvidia-smi`. Override when needed:

```
# Ampere, e.g. RTX 3080 Ti
make -C gpu double_fast -j GPU_ARCH=86

# Ada, e.g. RTX 4070
make -C gpu float_fast -j GPU_ARCH=89
```

1.4 EPYC Zen5, AVX-512, and AVX2

`scripts/detect_arch_flags.sh` is used by the Makefiles through `ARCH_FLAGS`. On a native machine, the safest high-performance option is usually:

```
make -C cpu clean
make -C cpu -j ARCH_FLAGS="-march=native -mtune=native"

make -C gpu clean
make -C gpu double_fast -j ARCH_FLAGS="-march=native -mtune=native"
```

For named AMD targets:

CPU target	GCC/Clang flags	Notes
EPYC Zen2	`ARCH_FLAGS="-march=znver2 -mtune=znver2"`	AVX2/FMA path
EPYC Zen3	`ARCH_FLAGS="-march=znver3 -mtune=znver3"`	AVX2/FMA path
EPYC Zen4	`ARCH_FLAGS="-march=znver4 -mtune=znver4"`	AVX-512 capable on GCC versions that support it
EPYC Zen5	`ARCH_FLAGS="-march=znver5 -mtune=znver5"`	Use with a compiler that knows `znver5`
Portable AVX2	`ARCH_FLAGS="-O3 -mavx2 -mfma"`	Runs on AVX2 machines
Explicit AVX-512	`ARCH_FLAGS="-O3 -mavx512f -mavx512dq -mavx512cd -mavx512bw -mavx512vl"`	Use only on machines that support these flags

There is no GCC/Clang option named `AVX12`. If the intended target is Zen5 with AVX-512, use `-march=znver5` when the compiler supports it. If the compiler is older, either upgrade GCC/Clang or use the explicit AVX-512 flags above after checking `lscpu | grep avx512`.

1.5 Legacy root Makefile

The root `Makefile` still exists for compatibility:

```
make                        # bin/mbs_po, CPU
make gpu_double_fast        # delegates to gpu/double_fast
make gpu_float_fast         # delegates to gpu/float_fast
make USE_CUDA=1             # legacy single-tree CUDA build
```

Prefer the split `cpu/` and `gpu/` builds for normal work.

2 Quick runs

```
# GPU double, oldauto orientation grid, full 0..180 degree output
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
    --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
    --grid 0 180 600 180 --threads 64 --close -o hex_gpu_double

# Same run, force CPU backend from the GPU binary
gpu/bin/mbs_po_gpu_double_fast --po --cpu -p 1 125.9 78.09 \
    --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
    --grid 0 180 600 180 --threads 64 --close -o hex_cpu_from_gpu_bin

# File particle scaled by equivalent size parameter
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat --k_eq 58.81 \
    --ri 1.6 0.002 -w 1.064 -n 14 --oldauto 2 --pole \
    --grid 0 180 600 180 --threads 16 --close -o particle_keq

# Two-point forward/backward check
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
    --ri 1.3116 0 -w 0.532 -n 1 --oldauto 2 --pole \
    --grid 0 180 600 1 --threads 64 --close -o check_0_180
```

3 Physical model

3.1 Pipeline

Stage	Main code path	Result
Particle setup	`src/particle`, `src/main.cpp`	Facets, normals, area, volume, symmetry
Geometrical tracing	`src/scattering`, `src/tracer`, `src/Splitting.cpp`	Output beams with direction, polygon, area, optical path, Jones matrix
PO diffraction	HandlerPO::ApplyDiffraction*, Handler::DiffractIncline*, CUDA equivalents	Far-field Jones amplitude per beam and direction
Coherent sum	`HandlerPO::AddToMueller`, GPU fused Mueller kernels	Sum Jones matrices over beams for one orientation
Mueller conversion	`src/math/Mueller.cpp`	4x4 Mueller matrix
Orientation average	`HandlerPOTotal`, `TracerPOTotal`	Final randomly oriented Mueller matrix

3.2 Ray Jones matrices

Each traced beam carries a 2x2 complex Jones matrix `beam.J`. Refraction and reflection multiply it by Fresnel coefficients in `Splitting.cpp`. The two polarization channels are the local vertical/horizontal basis of the incident and outgoing ray.

For a beam with local Jones matrix

```
J = [ Jvv  Jvh ]
    [ Jhv  Jhh ],
```

the code treats `J` as the coherent amplitude transform accumulated along the ray path. Cutoffs based on `|J|^2`, area, or `|J|^2*area` can remove weak beams before expensive diffraction.

3.3 Dyadic rotation into the scattering basis

Before adding a beam to a detector direction, the beam-local Jones matrix must be expressed in the detector polarization basis. `HandlerPO::RotateJones` and `RotateJonesFast` build this 2x2 rotation/projection using dot products between:

Vector	Meaning
`beam.Direction()`	Beam propagation direction after tracing
`direction`	Requested far-field scattering direction
`vf`	Forward reference direction
`info` polarization vectors	Precomputed beam polarization basis

The effective far-field Jones contribution has the structure used in `ApplyDiffractionFast`:

```
J_beam(theta, phi) = F_edge(theta, phi) * R_out(theta, phi) * F_n(theta, phi)
```

where:

Factor	Code	Meaning
`F_edge`	`DiffractInclineFast`, `DiffractIncline`, `DiffractInclineAbs`	Scalar Kirchhoff edge diffraction integral for the beam polygon
`R_out`	`RotateJones` or `KarczewskiJones`	Jones-basis rotation/projection from beam coordinates to scattering coordinates
`F_n`	`ComputeFnJones`	Fresnel/Jones correction for the beam normal and direction

The optional `--karczewski` flag replaces the default rotation with the Karczewski polarization matrix. The code comments note that `M11` is unchanged because the Frobenius norm is preserved, while polarization-sensitive elements can change.

3.4 Kirchhoff diffraction integral

For each traced output beam the illuminated aperture is a polygon. The PO far field is evaluated by a boundary/edge form of the Kirchhoff integral. In code this lives in:

Function	Use
Handler::DiffractIncline	CPU scalar non-absorbing edge integral
Handler::DiffractInclineFast	CPU optimized edge integral with precomputed edge data
Handler::DiffractInclineAbs	Absorbing variant
`GpuDiffraction.cu` kernels	CUDA implementation of the same aperture/edge calculation

Conceptually the scalar diffraction factor is the aperture phase integral

```
F(q) = integral_A exp(i k q.r) dA,
```

where `A` is the projected beam polygon, `k = 2*pi/lambda`, and `q` is the difference between the outgoing beam direction and the requested far-field direction. The implementation evaluates the polygon integral through its edges to avoid sampling the aperture area directly. For an edge from vertex `a` to `b`, the contribution is a stable complex exponential difference divided by the corresponding phase denominator; special small-denominator branches use sinc-like limits.

The GPU and CPU branches should use the same geometry, phase convention, and theta grid. Differences normally come from precision (`float` vs `double`), fast math, FFT interpolation, atomics/reduction order, and special pole or endpoint handling.

3.5 Coherent and incoherent Mueller accumulation

Default PO mode is coherent per orientation:

```
J_total(theta, phi) = sum_beams J_beam(theta, phi)
M(theta, phi)       = Mueller(J_total(theta, phi))
```

With `--incoh`, the conversion happens before beam summation:

```
M(theta, phi) = sum_beams Mueller(J_beam(theta, phi))
```

The Jones-to-Mueller conversion is implemented in `src/math/Mueller.cpp`. For Jones elements `S1..S4`, the code computes the standard 4x4 real Mueller matrix from bilinear products such as `|S1|^2`, `|S2|^2`, `Re(S1 conj(S2))`, and `Im(S1 conj(S2))`.

3.6 Orientation averaging

For randomly oriented particles, each orientation produces a Mueller matrix on the scattering grid. `HandlerPOTotal` and `TracerPOTotal` average orientations with the beta/gamma weights selected by the orientation mode. At beta poles, `--pole` uses one gamma value because all gamma rotations are equivalent at the exact pole. In current code, `--pole` keeps beta endpoints instead of midpoint beta sampling, so the beta pole is actually present in the grid.

3.7 Scattering-angle weights

Uniform theta grids output `Nth + 1` rows. `2pi*dcos` is the solid-angle ring weight assigned to the theta row:

```
dOmega(theta_j) = 2*pi * (cos(theta_left) - cos(theta_right))
```

For endpoint rows the interval is half-width and clipped to the requested range. For a full `0..180` grid the sum of `2pi*dcos` over all rows is `4*pi`.

4 Particles and sizes

Flag	Arguments	Description
`-p`	`TYPE PARAMETERS...`	Built-in particle. Exactly one of `-p` or `--pf` is required.
`--pf`	`FILE`	Load particle from file.
`--save-geometry`	`FILE`	Save the effective particle, after input scaling and geometry classification, in the native text format. Alias: `--save-particle`. The calculation then continues.
`--rs`	`SIZE`	Resize file particle to `Dmax = SIZE`. Mutually exclusive with `--k_eq`.
`--k_eq`	`X`	Resize so `k_eq = 2*pi*r_eq/lambda`. Requires wavelength.
`--ri`	`Re Im`	Complex refractive index. Absorption is enabled automatically when `Im != 0`, or explicitly with `--abs`.
`-w`	`LAMBDA`	Wavelength in micrometers.
`-n`	`N`	Maximum internal reflection/refraction depth.

Built-in particle types:

Type	Shape	Parameters
1	Hexagonal column/plate	`L D`: axial length and vertex-to-vertex diameter. The same type represents a column or a plate according to L/D.
2	Bullet	`L D`: total axial length and diameter. The pyramidal cap height is derived from the fixed 62° construction angle.
3	Bullet rosette	`L D [CAP]`: six bullets arranged as a rosette. If `CAP` is omitted, the same 62° construction is used.
4	Droxtal	`SCALE`: multiplies the fixed droxtal template. Legacy `L D SCALE` is accepted but `L,D` are ignored.
10	Concave hexagonal	`L D CAVITY_DEG`: column with top and bottom pyramidal cavities. The angle must be positive and less than $\arctan(L/D)$.

Type	Shape	Parameters
12	Two-column hexagonal aggregate	`L D 2`: exactly two equal eight-facet columns in the built-in relative placement. Use a file for a general aggregate.
999	Fixed built-in aggregate	`SCALE`: multiplies the fixed aggregate template used as a reproducible regression geometry.

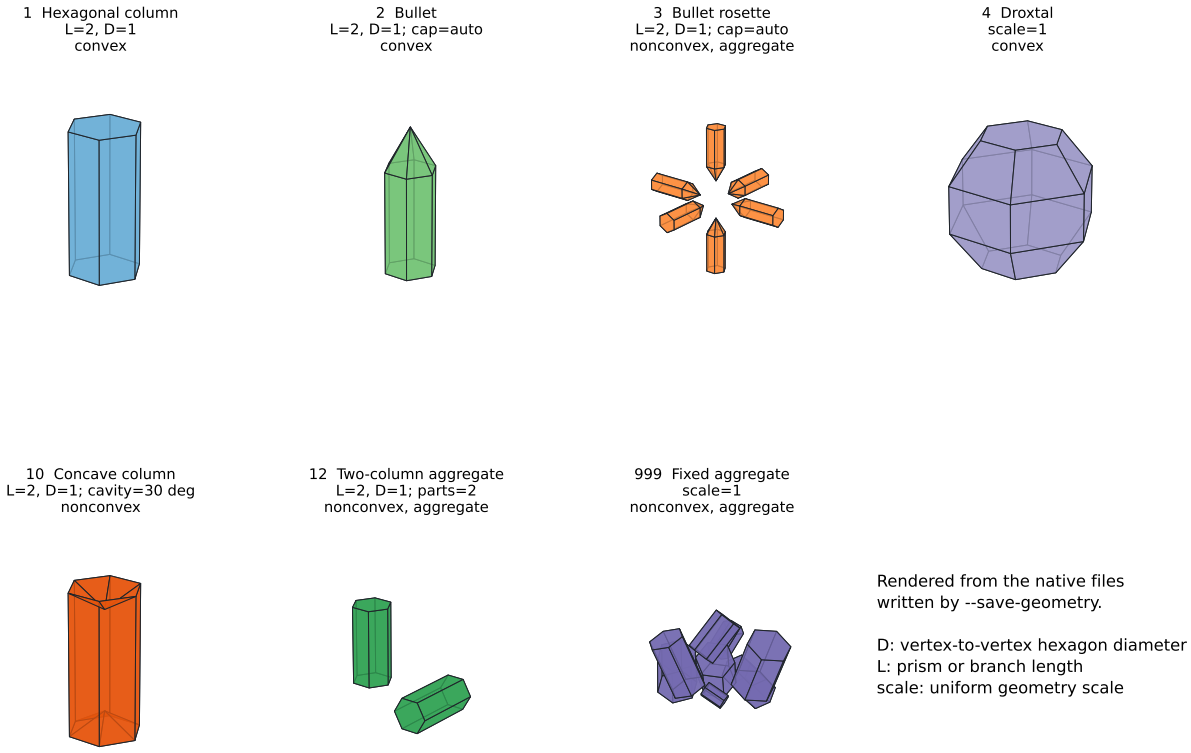

Figure 1: Geometries produced by the current built-in constructors. The images are rendered from the native files in `examples/particles`, not from separate drawing models.

4.1 Native particle file and geometry export

The native format starts with three nonempty header records:

```
CONCAVE
AGGREGATE [FACETS_PER_PART]
BETA_DOMAIN_DEG GAMMA_DOMAIN_DEG

x0 y0 z0
x1 y1 z1
x2 y2 z2

x0 y0 z0      # next facet
...
```

`CONCAVE` is 0 or 1. `AGGREGATE` is 0 for one object, or `1 FACETS_PER_PART` for equal-size aggregate components. The third record gives the positive beta and gamma fundamental-domain widths. Blank lines separate facets; each facet must have at least three finite, non-collinear vertices. Full-line and inline comments begin with `#`. Vertex order defines the facet normal, so a hand-edited file must preserve one consistent winding convention.

`--save-geometry FILE` writes 17-digit coordinates and validates aggregate metadata before writing. A useful round-trip check is:

```
gpu/bin/mbs_po_gpu_double_fast --po -p 10 20 12 15 \
    --save-geometry concave.particle ...
gpu/bin/mbs_po_gpu_double_fast --po --pf concave.particle ...
```

The exported file contains the effective scaled geometry. It is therefore the appropriate artifact for reproducing a run or inspecting the exact facets used by clipping. The automatically written `particle_for_check.dat` remains available in result directories; the explicit flag is preferable when a stable path is needed.

Equivalent-size scaling:

```
r_eq = (3 V / (4*pi))^(1/3)
k_eq = 2*pi*r_eq/lambda
scale = (k_eq_target * lambda / (2*pi)) / r_eq_original
```

5 Orientation grids

Mode	Arguments	Best use	Notes
`--oldauto`	`DIV`	Production regular grid	Grid step follows diffraction-limited angular scale; common values are 2, 4, 8.
`--random`	`Nb Ng`	Manual beta/gamma regular grid	Uses symmetry-reduced beta/gamma domain unless overridden.
`--fixed`	`BETA GAMMA`	Single-orientation debugging	Angles are degrees.
`--orientfile`	`FILE`	Reproducible custom orientations	One beta/gamma pair per line.
`--sobol`	`N`	Quasi-random orientation average	Good convergence for broad scans.
`--sobol_seed`	`N S`	Seeded Sobol/Owen sequence	Useful for repeatable convergence checks.
`--sobol_ring`	`Nb Ng`	Sobol beta with uniform gamma rings	Hybrid orientation grid.
`--so3_quat`	`N`	Full SO(3) Hammersley quaternions	Does not rely on beta/gamma symmetry domain.
`--hammersley`	`N`	Low-discrepancy orientations	Debug/experimental.
`--lattice`	`N`	Rank-1 lattice orientations	Debug/experimental.
`--lattice_z`	`N Z`	Rank-1 lattice with explicit generator	Debug/experimental.
`--euler_quad`	`Nb Ng`	Gauss in cos(beta), periodic gamma	High-order quadrature.
`--euler_adapt`	`Nb NgMax`	Adaptive gamma count per beta ring	Reduces work near beta poles.
`--adaptive_euler`	`EPS`	Converged regular Euler grid	Refines N_β and N_γ separately, then together.
`--adaptive`	`EPS`	Adaptive Sobol count only	Doubles N; it does not silently alter theta, phi, or reflection depth.
`--auto`	`EPS`	Auto theta, phi, orientation count	Keeps the requested reflection depth fixed.
`--autofull`	`EPS`	Auto n, theta, phi, orientations	Slower, more complete search.
`--oldautofull`	`EPS`	Autofull with converged Euler grid	Independently converges N_β and N_γ.
`--adaptive-config`	`FILE`	Reproducible adaptive settings	Per-parameter minimum, maximum, and tolerance.

Orientation modifiers:

Flag	Arguments	Description
`--ring_points`	`N`	Points per diffraction ring for oldauto estimates; default 3.
`--mirror_gamma`	none	Halve the gamma symmetry range and mirror the Mueller result.
`--sym`	`Sb Sg`	Override symmetry domain: beta range is `pi/Sb`, gamma range is `2*pi/Sg`.
`--b`	`B1 B2`	Manual beta range in degrees for `--random`.
`--g`	`G1 G2`	Manual gamma range in degrees for `--random`.
`--maxorient`	`N`	Maximum orientations for adaptive modes.
`--max_beta_points`	`N`	Hard adaptive N_β limit.
`--max_gamma_points`	`N`	Hard adaptive N_γ limit.
`--chunk`	`N`	Orientation/gamma chunk size.
`--coh_orient`	none	Legacy coherent-across-orientations mode.
`--pole`	none	Use one gamma value at beta poles. Best with endpoint grids such as `--oldauto`.
`--owen_avg`	`K`	Average K nested Owen final seeds in `--autofull`; default can be controlled by environment.
`--owen_seeds`	`S...`	Explicit Owen seeds for `--autofull` final averaging.

6 Scattering grids

Flag	Arguments	Description
`--grid`	`T1 T2 Nphi Nth`	Uniform theta grid from `T1` to `T2` degrees. Output has `Nth + 1` theta rows.
`--grid`	`R Nphi Nth`	Backscatter cone grid of radius `R` degrees.
`--tgrid`	`FILE`	Non-uniform theta grid in degrees, one row per theta value.
`--auto_tgrid`	`EPS`	Adaptive theta grid by bisection of measured full-Mueller interpolation error; no fixed halo angles are used.
`--auto_phi`	none	Compatibility form of measured phi convergence with tolerance 0.02.
`--adaptive_phi`	`EPS`	Converge N_ϕ only.
`--adaptive_reflections`	`EPS`	Converge internal reflection depth n only.
`--nphi`	`N`	Override phi count. Highest priority for phi.
`--diffraction-limit-grid`	`FACTOR`	Derive uniform theta and equatorial-phi steps from the diffraction estimate.
`--latitude-phi-grid`	none	Reduce the phi count in proportion to $\sin\theta$ away from the equator.
`--max_theta_points`	`N`	Hard theta-point limit; default 4097.
`--max_phi_points`	`N`	Hard N_ϕ limit; default 2400.
`--convergence_passes`	`N`	Consecutive passing refinements required; default 2.
`--filter`	`DEG`	Restrict output to a backscattering cone. Legacy/debug use.
`--point`	none	Backscatter point mode. Legacy; avoid for optimized regular-grid production.

6.1 Diffraction-limit and latitude-phi grids

For a PO `--diffraction-grid DIV` run, `--diffraction-limit-grid FACTOR` derives the scattering grid from the current particle maximum dimension $l_{\max}$ and wavelength λ:

$$\xi = \text{FACTOR}\, 0.69 \frac{\lambda}{l_{\max}}, \qquad N_\theta = \left\lceil \frac{\pi}{\xi} \right\rceil,$$

$$N_{\phi,\text{eq}} = \text{round\_up}_6 \left[\max\left(12, \left\lceil \frac{2\pi}{\xi} \right\rceil \right) \right].$$

The output contains $N_\theta + 1$ rows from zero to π. `FACTOR=1` uses the estimate literally; `FACTOR=0.5` approximately halves both angular steps, while a factor above one makes the grid coarser. This is an a-priori resolution estimate, not an a-posteriori accuracy guarantee. Validate a production factor against a finer value for every required Mueller element.

With `--latitude-phi-grid`, each theta row uses

$$N_\phi(\theta) = \min\left\{ N_{\phi,\mathrm{eq}}, \mathrm{round\_up}_6\left[\max\left(12, \left\lceil \frac{2\pi \sin\theta}{\xi} \right\rceil\right)\right]\right\}.$$

Equatorial spacing is unchanged while redundant azimuth samples are removed near the poles. Counts are rounded to a multiple of six, and a pole shares its neighboring work group for stable basis handling. Accumulation still uses the full rectangular $N_{\phi,\mathrm{eq}}$ output layout, so the file format does not change.

Both options require `--method po --diffraction-grid DIV` and one particle size per process. The diffraction-limit option conflicts with explicit `--nphi`; the latitude grid is not supported by a shared serial multi-size trace. Run each size in its own process so $l_{\max}$ and the grid are recomputed.

```
CUDA_VISIBLE_DEVICES=0,1,2,3 \
MBS_GPU_GROUPS=1 MBS_GPU_MULTI_MAX=4 \
gpu/bin/mbs_po_gpu_double ... --diffraction-limit-grid 0.5 --latitude-phi-grid
```

Selection rules:

Quantity	Rule
Theta grid	Exactly one of `--tgrid`, `--grid`, or `--auto_tgrid`; unified auto modes treat an explicit grid as fixed.
Phi grid	Standalone adaptive phi selection and explicit `--nphi` are mutually exclusive. Unified auto modes treat explicit `--nphi` as fixed and tune the remaining dimensions.

6.2 Unified convergence criterion

For two approximations A and B, phi, reflection, and Euler candidate comparisons use the row error

$$e_{\mathrm{row}} = \max\left(\frac{|M_{11}^A - M_{11}^B|}{\max(|M_{11}^A|, |M_{11}^B|, \epsilon_0)}, \max_{(i,j)\neq(1,1)} \frac{|R_{ij}^A - R_{ij}^B|}{\max(1, |R_{ij}^A|, |R_{ij}^B|)} \right), \qquad R_{ij} = \frac{M_{ij}}{M_{11}}.$$

Here M_{ij} is a Mueller element, R_{ij} is its normalized value, and ϵ_0 protects a vanishing M_{11}. The maximum over theta rows is combined with the relative change of $\int M_{11}\, d\Omega$. Reflection-depth tests also include outgoing energy. Theta convergence is different: it evaluates the actual Mueller row at one-third and two-thirds of every interval against interpolation and checks the same integrated M_{11}. Sobol orientation convergence checks incoming energy and all 16 Mueller elements on control-theta rows selected from the actual scattering grid.

The parameters are not strictly independent. Reflection depth changes the beam tree; phi changes the azimuth average used to expose theta structure; and orientation sampling changes every averaged row. Therefore the joint order is

$$n \longrightarrow N_\phi \longrightarrow \{\theta_j\} \longrightarrow N_{\mathrm{orient}}.$$

The `--auto` modes repeat the complete sequence on a common larger pilot set until the pilot implied by the selected orientation count and all selected values form a coupled fixed point. Regular Euler convergence alternates N_β and N_γ, then doubles both together. Reaching any hard limit in phi, reflection, theta, Euler, or a unified mode is reported as an error rather than accepted as convergence. Standalone `--adaptive-orientations` is exploratory: at its orientation cap it warns and writes the best result; unified modes use strict orientation convergence. The configured passing streak applies to phi, reflection, Euler, and orientation searches, while theta uses interval refinement directly. Every trial is written to `<output>/<name>_convergence.tsv`.

6.3 Adaptive configuration file

Use `--adaptive-config FILE` to keep production convergence settings outside a long command line. The documented template is `configs/adaptive.example.conf`. It uses strict `key = value` assignments and accepts inline comments after `#` or `;`.

The file defines separate minimum, maximum, and relative tolerance values for reflection depth, laboratory alpha (internally N_ϕ), theta points, Sobol orientations, beta points, and gamma points. It also defines the number of consecutive passing refinements and the pilot and sweep limits of the coupled controller. A tolerance of `0.01` means one percent.

```
mbs_po --autofull 0.02 --adaptive-config configs/adaptive.example.conf ...
```

Per-parameter tolerances in the file replace the mode-wide `EPS`; a missing tolerance inherits `EPS`. Explicit command-line `--max-*`, `--max-reflections`, and `--convergence-passes` values override the file. Unknown keys, duplicates, invalid ranges, non-power-of-two Sobol limits, and malformed lines are rejected before the calculation, with the file line and a corrective hint.

6.4 Why column symmetry does not divide Nphi

The code uses the Euler convention

$$\mathbf{R} = R_z(\alpha)R_y(\beta)R_z(\gamma).$$

The scattering-azimuth loop is the rotationally covariant implementation of the laboratory-angle α average, so $N_\phi = N_\alpha$ in this context. A hexagonal column has the body-axis symmetry $R_z(2\pi/6)$; it reduces the γ domain to $0 \le \gamma < \pi/3$. For $\beta \neq 0, \pi$, it does not make the tilted particle invariant under a laboratory $R_z(2\pi/6)$ rotation, hence $0 \le \alpha < 2\pi$ is still required. An automatic division of N_ϕ by six would therefore be incorrect. The aliases `--alpha-points` and `--adaptive-alpha` only clarify the meaning of the existing phi controls; they do not enable an approximation.

For full angular integrals use `--grid 0 180 Nphi Nth`. Endpoint rows are physical half-cells; they must not have zero weight.

7 Beam and trace cutoffs

Cutoffs act at two different stages. Output-beam cutoffs discard a completed ray branch before PO diffraction. Trace cutoffs stop an internal branch while the reflection/refraction tree is being built. With reference values taken from the current orientation, the three dimensionless tests are

$$j_b = \frac{\|J_b\|_F^2}{J_{\text{ref}}^2}, \qquad a_b = \frac{A_b}{A_{\text{ref}}}, \qquad i_b = j_b a_b.$$

Here J_b is the accumulated Jones matrix of branch b, A_b is its polygon area, and i_b is a simple intensity-area importance estimate. When several tests are enabled, rejection uses logical *OR*: satisfying any one smallness test removes the branch. Reason counters can therefore overlap.

7.1 Coherent presets

`--cutoff-profile MODE` configures all relative thresholds and the small-fragment ratio together. It cannot be combined with an individual cutoff or `--beam-area-ratio`.

Profile	Output i_b	Trace i_b	Area ratio	Intended use
`off`	0	0	disabled	Reference and convergence runs; no configurable cutoff approximation.
`safe`	10^{-10}	10^{-12}	10^{8}	First production candidate after comparison with `off`.
`balanced`	10^{-8}	10^{-10}	10^{5}	Faster scans after particle-class calibration.

Profile	Output i_b	Trace i_b	Area ratio	Intended use
`fast`	10^{-6}	10^{-8}	10^3	Exploratory scans; not a validation setting.
legacy default	0	0	100	Backward-compatible behavior when no profile or individual cutoff is supplied.

The preset values enable the combined importance criterion, not separate Jones and area OR thresholds. This avoids removing a large but weak branch or a small but intense branch solely because one factor is small. The legacy area-ratio simplification remains at 100 for reproducibility; use `--cutoff-profile off` when a genuinely uncut reference is required. The profiles do not set `--trace-max-beams`.

7.2 Individual controls

Flag	Range	Exact action
`--beam-cutoff EPS`	$[0, 1]$	Legacy shortcut setting both output Jones and output area thresholds; a beam is rejected if either test passes.
`--beam-cutoff-jones EPS`	$[0, 1]$	Output test $j_b <$ EPS.
`--beam-cutoff-area EPS`	$[0, 1]$	Output test $a_b <$ EPS.
`--beam-cutoff-importance EPS`	$[0, 1]$	Output test $i_b <$ EPS.
`--trace-cutoff EPS`	$[0, 1]$	Legacy shortcut setting both internal Jones and area thresholds; pruning uses OR.
`--trace-cutoff-jones EPS`	$[0, 1]$	Internal test $j_b <$ EPS.
`--trace-cutoff-area EPS`	$[0, 1]$	Internal test $a_b <$ EPS.
`--trace-cutoff-importance EPS`	$[0, 1]$	Internal test $i_b <$ EPS.
`--beam-area-ratio R`	$R \geq 1$	After a nonconvex split, replace two fragments by the larger one only when their area ratio is at least R.
`--trace-max-beams N`	$N \geq 0$	Hard per-orientation beam-tree cap; 0 disables the configured cap. Any reported hit means that branches were truncated and the run is not a cutoff-convergence reference.

If a legacy common shortcut and a corresponding specific Jones/area option are supplied together, the specific value overrides that side and the program prints a warning. A profile instead rejects such mixing as an input error, so a command cannot silently become a partly modified preset.

Every completed run appends `OUTPUT BEAM CUTOFF REPORT` and `TRACE CUTOFF REPORT` blocks to the log. They contain the selected profile, effective thresholds, candidate/kept/rejected counts, overlapping reason counts, fragment simplifications, configured-cap hits, and hard internal tree-limit hits. Calibrate a profile by running the same geometry, orientation set, n, theta grid, and N_ϕ with `off` and with the candidate profile; compare all 16 M_{ij}/M_{11}, not only runtime or the rejected-beam percentage.

8 GPU and multi-GPU

8.1 CUDA backend

Flag	Description
`--gpu`	Use CUDA diffraction backend. Optional in `gpu/` binaries because they default to GPU.
`--cpu`	Force CPU backend from a GPU-capable binary.
`--fft`	Use cuFFT angular phi interpolation backend. Requires GPU. The direct-phi compression factor is selected automatically.
`--fft-factor N`	Enable FFT interpolation with an explicit compression factor $1 \leq N \leq 64$. $N = 1$ keeps the direct phi grid and is useful as a same-backend reference.

Flag	Description
`--fft-tolerance EPS`	Enable a nested-grid error estimate and one automatic refinement when the estimate exceeds $0 < \text{EPS} < 1$.
`--threads N`	OpenMP host worker threads. Also controls tracing-side parallelism before GPU diffraction.

8.2 FFT phi compression and error control

Let N_ϕ be the requested output azimuth count and f the compression factor. The first CUDA pass evaluates approximately

$$N_{\phi,\text{direct}} = \max\left(16, \left\lfloor \frac{N_\phi}{f} \right\rfloor\right)$$

direct samples, then uses periodic Fourier zero-padding to reconstruct the N_ϕ output grid. For $f = 1$, $N_\phi < 32$, or a factor that cannot reduce the grid, the implementation uses the direct path. The speed gain is therefore workload dependent and is bounded by the fraction of runtime spent in phi-direction diffraction; tracing itself is not reduced.

`--fft` asks the implementation to select f. `--fft-factor N` makes the requested compression explicit and has priority over `MBS_FFT_PHI_FACTOR`. Large factors are not an accuracy promise: sharp azimuthal structure has high Fourier modes and can be smoothed or aliased.

With `--fft-tolerance EPS`, the code also evaluates a nested grid with

$$N_{\phi,2} = \min(N_\phi, 2N_{\phi,\text{direct}}).$$

It compares significant M_{11} samples and all 16 Mueller elements using an M_{11}-based scale. The reported estimate is

$$\hat{e} = \max\left(\max_{M_{11}\ \text{significant}} \frac{|M_{11}^{(1)} - M_{11}^{(2)}|}{|M_{11}^{(2)}|}, \max_{i,j} \frac{|M_{ij}^{(1)} - M_{ij}^{(2)}|}{\max(|M_{11}^{(2)}|, 10^{-3} M_{11,\max}^{(2)})} \right).$$

If $\hat{e} >$ EPS, the result reconstructed from the doubled direct grid is retained. This is a one-step a-posteriori estimator, not a rigorous bound and not an iterative convergence proof. It costs an additional diffraction pass and temporarily holds coarse, refined, and output arrays. For a final reference, omit all FFT flags or use `--fft-factor 1`; for production, first calibrate f and EPS on representative sizes and particle shapes.

```
# Explicit 4x compression, refine once when the estimate exceeds 1 percent
gpu/bin/mbs_po_gpu_float_fast ... --fft-factor 4 --fft-tolerance 0.01
```

The final `FFT INTERPOLATION REPORT` records the requested and effective factor, direct and output N_ϕ, number of FFT-path and validation calls, number of refined calls, and the worst nested-grid estimate. Factor 1 is labelled as direct/no interpolation. The fused multi-`k_eq` FFT path does not perform this validation; when tolerance or the diagnostic check is requested, the dispatcher uses a supported fallback instead of silently skipping the check.

GPU runtime errors are fatal in the split GPU build. For debugging only:

```
MBS_GPU_ALLOW_FALLBACK=1 gpu/bin/mbs_po_gpu_float_fast ...
```

8.3 Why the GPU version is faster

The expensive PO part is the repeated evaluation of the same beam aperture integral for many detector directions and many orientations. The CPU traces rays and prepares compact beam records; the GPU then evaluates diffraction and, in the fast paths, Mueller accumulation on a large regular grid.

Work item	CPU path	GPU path	Parallel granularity
Ray tracing through facets	OpenMP over orientations/gamma blocks	Mostly still CPU-side; `--gpu_trace` is only a candidate prefilter	Orientation/chunk level
Beam packing	CPU host code	CPU prepares `GpuBeam`/packed beam buffers and copies to device	Beam records
Edge diffraction	Scalar/vector CPU loops	CUDA kernels in `GpuDiffraction.cu`	Beam x theta x phi x orientation
Jones coherent sum	CPU arrays of complex 2x2 matrices	Atomic or no-atomic device kernels	Grid cell and orientation
Jones to Mueller	CPU after Jones sum	Separate `mueller_batch_kernel` or fused in diffraction kernel	Grid cell
Multi-`k_eq` scans	Repeat diffraction per size	Optional fused multi-`k_eq` CUDA pass	Size x beam x grid
FFT phi interpolation	CPU direct phi grid unless disabled	cuFFT low-phi pass plus zero-padding reconstruction	Theta/orientation batches

The main speedup comes from exposing a very large product of independent operations:

```
Nwork ~= Norient * Nbeams_per_orientation * Ntheta * Nphi
```

Each beam/direction contribution is mostly independent until the coherent Jones sum. That makes the diffraction stage a good CUDA workload: many threads do the same edge-integral arithmetic over different `(orientation, beam, theta, phi)` combinations.

8.4 Atomic and no-atomic GPU accumulation

There are two families of CUDA accumulation paths in `src/cuda/GpuDiffraction.cu`.

Mode	Main kernels	How accumulation works	When useful
Atomic Jones accumulation	diffraction_kernel	Each thread computes one beam/grid contribution and uses `atomicAdd` into the complex Jones buffer. Afterwards mueller_batch_kernel converts the summed Jones matrix to Mueller.	General fallback, supports more combinations such as no-shadow output.
No-atomic grid kernels	diffraction_grid_kernel	Work is reorganized so a thread owns a grid/output slot and loops over beams, avoiding multiple writers to the same Jones cell.	Full-only output when memory/layout allows it.
Fused Mueller kernels	diffraction_grid_mueller _kernel, diffraction_grid _mueller_full_kernel, *_full8_kernel, *_mixed8_kernel	The kernel computes coherent Jones locally for an output slot and immediately converts/adds Mueller, avoiding large intermediate Jones buffers.	Fast production path for full-only GPU runs.
Staged orientation Mueller	diffraction_grid_mueller _orient_kernel + reduce _mueller_orient_kernel	First writes per-orientation Mueller, then reduces orientations.	Reduces contention and gives a deterministic reduction shape for large orientation batches.
Multi-`k_eq` fused	diffraction_grid_mueller _multik_kernel	Computes several nearby equivalent sizes from one packed beam batch.	Shared-batch `--multikeq` scans.

In the atomic path the atomics are on the real and imaginary components of the 2x2 Jones matrix:

J00.re, J00.im, J01.re, J01.im,
J10.re, J10.im, J11.re, J11.im

For _noshadow output the same set of atomics is also applied to the no-shadow Jones buffer. This is correct for coherent PO because beams must be summed as Jones amplitudes before conversion to Mueller. The cost is contention when many beams hit the same detector cell.

The no-atomic/fused paths avoid that contention by changing ownership: one CUDA thread or thread group owns an output grid element and accumulates the beams for that element in registers/local variables. This is often faster for full Mueller output because it reduces global-memory atomics and may skip the large intermediate Jones buffer. The tradeoff is that these paths are more specialized and may be disabled for diagnostic modes that require extra outputs.

Useful controls:

Environment variable	Effect
MBS_GPU_NO_ATOMICS=1	Force no-atomic path when supported.
MBS_GPU_NO_ATOMICS=0	Force atomic path for comparison/debugging.
MBS_GPU_FUSED_MUELLER=1	Prefer fused diffraction-to-Mueller kernels when no-atomic mode is active.
MBS_GPU_STAGE_MUELLER=1	Use staged per-orientation Mueller plus reduction where supported.
MBS_GPU_NO_VERTEX_CACHE=1	Disable cached/packed vertex path for debugging.
MBS_GPU_TIMING=1	Print GPU timing breakdown for count/pack/copy/kernels/d2h/add.
MBS_GPU_BLOCK=N	Override CUDA block size for kernel tuning.

Do not confuse this with full GPU ray tracing. The production GPU backend is primarily a diffraction and Mueller-accumulation accelerator. --gpu_trace only prefilters nonconvex tracing candidates; exact intersections remain CPU-side.

For a variable latitude-phi grid, MBS_GPU_GROUPS=1 enables a dedicated exact scheduler. Independent theta-row work groups are assigned dynamically to visible GPUs. Orientation tracing and prepared beams remain shared in host memory, while each device owns one CUDA workspace. Packed beams, weights, and offsets are uploaded once per orientation chunk per device and reused for subsequent theta groups handled by that worker.

Use CUDA_VISIBLE_DEVICES to select devices. Use MBS_GPU_MULTI=N or MBS_GPU_MULTI_MAX=N to limit the number of workers; MBS_GPU_MULTI=0 leaves one GPU. The scheduler requires direct CUDA diffraction; FFT interpolation and theta-zone beam filtering retain the existing path. Speedup need not be linear because CPU tracing, packing, PCIe transfers, final reduction, and theta-group imbalance remain in wall time.

8.5 Multi-size scans

Flag	Arguments	Description
--multigrid	Dmin Dmax N	Log-spaced scan in absolute maximum particle dimension; each value uses $x = \pi D_{\max}/\lambda$, independent of the source size.
--multikeq	Kmin Kmax N	Log-spaced scan in equivalent size parameter.
--multikeq_list	FILE	Exact list of k_eq values, one per line.
--multigrid_parallel	N	Run scan points as child processes; 0 means auto.
--multigrid_threads	N	OpenMP threads per child process.
--gpu_devices	LIST	Comma-separated CUDA device list, for example 0,1,2,3,4.
--multikeq_shared_batches	none	Batch nearby k_eq values per GPU child and reuse tracing from the largest value in the batch.
--multikeq_batch_ratio	R	Maximum kmax/kmin inside one shared batch; default 1.05.

Exact multi-GPU scan, one process per active GPU slot:

```
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat \
    --multikeq_list keq.txt --ri 1.6 0.002 -w 1.064 -n 12 \
```

```
    --oldauto 2 --grid 0 180 600 180 --fft --close \
    --multigrid_parallel 0 --multigrid_threads 16 --gpu_devices 0,1,2,3,4 \
    -o scan_exact
```

Shared-batch scan:

```
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat \
    --multikeq_list keq.txt --ri 1.6 0.002 -w 1.064 -n 12 \
    --oldauto 2 --grid 0 180 600 180 --fft --close \
    --multigrid_parallel 0 --multigrid_threads 16 --gpu_devices 0,1,2,3,4 \
    --multikeq_shared_batches --multikeq_batch_ratio 1.05 \
    -o scan_shared
```

Experimental fused multi-k_eq GPU diffraction:

```
MBS_GPU_MULTI_K_FULL=1 gpu/bin/mbs_po_gpu_float_fast ...
```

Use tighter `--multikeq_batch_ratio` for accuracy; use exact mode when each size must have its own oldauto reference grid.

9 All flags

9.1 Method, particle, and physical parameters

Flag	Arguments	Description
`--po`	none	Physical Optics mode.
`--go`	none	Geometrical Optics mode.
`-p`	`TYPE L D [extra]`	Built-in particle.
`--pf`	`FILE`	Particle from file.
`--save-geometry`	`FILE`	Save the effective native particle and continue; alias `--save-particle`.
`--rs`	`SIZE`	Resize file particle by Dmax.
`--k_eq`	`X`	Resize by equivalent size parameter.
`--ri`	`Re Im`	Refractive index.
`-w`	`LAMBDA`	Wavelength in micrometers.
`-n`	`N`	Maximum internal reflections/refractions.
`--abs`	none	Enable absorption accounting. Also enabled automatically for nonzero `Im(ri)`.
`--abs_points`	`N` or `all`	Absorption sampling: center point or all polygon vertices.

9.2 Orientation flags

Flag	Arguments	Description
`--fixed`	`BETA GAMMA`	Single orientation in degrees.
`--random`	`Nb Ng`	Regular beta/gamma grid.
`--sobol`	`N`	Sobol orientations.
`--so3_quat`	`N`	Full SO(3) quaternion orientations.
`--sobol_seed`	`N S`	Sobol with explicit Owen seed.
`--sobol_ring`	`Nb Ng`	Sobol beta with gamma rings.
`--hammersley`	`N`	Hammersley orientation set.
`--lattice`	`N`	Rank-1 lattice orientation set.
`--lattice_z`	`N Z`	Rank-1 lattice with explicit generator.
`--euler_quad`	`Nb Ng`	Gauss beta x periodic gamma quadrature.
`--euler_adapt`	`Nb NgMax`	Gauss beta with adaptive gamma count.
`--adaptive_euler`	`EPS`	Independently converge N_β and N_γ, then verify them jointly.
`--montecarlo`	`N`	Pseudo-random Monte Carlo orientations.
`--adaptive`	`EPS`	Adaptive Sobol orientation count only.
`--auto`	`EPS`	Auto theta, phi, and orientation count.
`--autofull`	`EPS`	Auto n, theta, phi, and orientations.

Flag	Arguments	Description
`--oldautofull`	`EPS`	Autofull search with a converged regular Euler final grid.
`--adaptive-config`	`FILE`	Load per-parameter adaptive ranges and tolerances.
`--owen_avg`	`K`	Average final Owen seeds.
`--owen_seeds`	`S...`	Explicit final Owen seeds.
`--oldauto`	`DIV`	Physics-based regular grid.
`--ring_points`	`N`	Points per diffraction ring estimate.
`--mirror_gamma`	none	Use mirrored half gamma domain.
`--orientfile`	`FILE`	Load beta/gamma orientations from file.
`--b`	`B1 B2`	Beta range for `--random`.
`--g`	`G1 G2`	Gamma range for `--random`.
`--maxorient`	`N`	Maximum adaptive orientation count.
`--max_beta_points`	`N`	Maximum adaptive N_β.
`--max_gamma_points`	`N`	Maximum adaptive N_γ.
`--chunk`	`N`	Chunk size for orientation/gamma processing.
`--coh_orient`	none	Legacy coherent orientation mode.
`--pole`	none	Exact beta-pole gamma shortcut.
`--sym`	`Sb Sg`	Override particle symmetry.

9.3 Scattering grid and acceleration flags

Flag	Arguments	Description
`--grid`	`T1 T2 Nphi Nth`	Uniform theta range.
`--grid`	`R Nphi Nth`	Backscatter cone.
`--tgrid`	`FILE`	Non-uniform theta grid.
`--auto_tgrid`	`EPS`	Adaptive theta grid.
`--auto_phi`	none	Measured phi convergence at tolerance 0.02.
`--adaptive_phi`	`EPS`	Converge N_ϕ only.
`--adaptive_reflections`	`EPS`	Converge n only.
`--nphi`	`N`	Override phi count.
`--diffraction-limit-grid`	`FACTOR`	Set theta and equatorial-phi steps to FACTOR $0.69\lambda/l_{\max}$.
`--latitude-phi-grid`	none	Use a row-dependent phi count proportional to $\sin\theta$.
`--max_theta_points`	`N`	Adaptive theta limit.
`--max_phi_points`	`N`	Adaptive phi limit.
`--convergence_passes`	`N`	Required passing streak.
`--threads`	`N`	OpenMP worker threads.
`--gpu`	none	CUDA backend.
`--cpu`	none	Force CPU backend in GPU build.
`--fft`	none	cuFFT phi interpolation with automatic compression.
`--fft-factor`	`N`	Explicit direct/output phi compression, $1 \ldots 64$; also enables FFT.
`--fft-tolerance`	`EPS`	Nested-grid estimate in $0 <$ EPS < 1 with one doubled-direct-grid refinement.
`--cutoff-profile`	`MODE`	Complete preset: `off`, `safe`, `balanced`, or `fast`.
`--beam-cutoff`	`EPS`	Legacy output shortcut; reject when relative Jones OR area is below EPS.
`--beam-cutoff-jones`	`EPS`	Output cutoff by relative squared Jones magnitude.
`--beam-cutoff-area`	`EPS`	Output cutoff by relative beam area.
`--beam-cutoff-importance`	`EPS`	Output cutoff by relative Jones intensity times area.
`--trace-cutoff`	`EPS`	Legacy internal shortcut; prune when relative Jones OR area is below EPS.
`--trace-cutoff-jones`	`EPS`	Prune trace tree by relative squared Jones magnitude.
`--trace-cutoff-area`	`EPS`	Prune trace tree by relative area.
`--trace-cutoff-importance`	`EPS`	Prune trace tree by relative Jones intensity times area.

Flag	Arguments	Description
`--trace-max-beams`	`N`	Hard per-orientation beam-tree cap; `0` disables the configured cap.
`--gpu_trace`	none	Experimental CUDA prefilter for nonconvex tracing candidates.
`--trace_prefilter`	none	Enable CPU projected-AABB prefilter.
`--no_trace_prefilter`	none	Disable CPU projected-AABB prefilter.
--trace_prefilter_margin	`M`	AABB prefilter margin.
`--trace_prefilter_stats`	none	Print prefilter counters.
`-r`, `--beam-area-ratio`	`RATIO`	Small-fragment area ratio, at least 1; legacy default `100`.

9.4 Output, diagnostics, and legacy flags

Flag	Arguments	Description
`-o`	`NAME`	Output path/name.
`--close`	none	Exit after computation.
`--log`	`SEC`	Progress logging interval.
`--checkpoint`	none	Save/resume long `--orientfile`, `--oldauto`, and `--random` runs.
`--save_betas`	none	Save per-beta Mueller files.
`--full_only`	none	Write only full Mueller output. This is the default.
`--noshadow_output`	none	Also write `_noshadow` Mueller output.
`--shadow`	none	Legacy flag; currently no effect.
`--shadow_off`	none	Disable shadow beam. Diagnostic use.
`--incoh`	none	Incoherent per-beam Mueller sum.
`--jones`	none	Write Jones matrices where supported.
`--karczewski`	none	Use Karczewski polarization matrix.
`--legacy_sign`	none	Use old Fresnel sign convention.
`--ot_phase_avg`	none	Average optical-theorem extinction over one far-reference phase period.
`--ot_phase_shift`	`F`	Diagnostic optical-theorem phase shift in wavelengths.
`--ot_ping`	`D`	Legacy far-screen optical-theorem phase rotation.
`--filter`	`DEG`	Restrict output to backscatter cone.
`--point`	none	Legacy backscatter point mode.
`--tr`	`FILE`	Load trajectory file.
`--all`	none	Calculate all loaded trajectories.
`--gr`	none	Output trajectory groups.
`--forced_convex`	none	Force convex processing.
`--forced_nonconvex`	none	Force nonconvex processing.
`--help`, `-h`	none	Print help.
`--help-debug`	none	Print full debug/experimental help.

10 Environment variables

Production-use variables:

Variable	Meaning
`CUDA_VISIBLE_DEVICES`	Standard CUDA device visibility control.
`OMP_NUM_THREADS`	OpenMP default thread count when `--threads` is not used.
`MBS_GPU_ALLOW_FALLBACK=1`	Allow old GPU-to-CPU fallback after CUDA failure. Debug only.
`MBS_GPU_MEM_FRACTION`	Fraction of free GPU memory allowed for internal buffers.
`MBS_GPU_NO_ATOMICS`	Select no-atomic (`1`) or atomic (`0`) GPU accumulation path when possible.
`MBS_GPU_FUSED_MUELLER`	Select fused diffraction-to-Mueller kernels when no-atomic mode is active.
`MBS_GPU_STAGE_MUELLER`	Enable staged per-orientation Mueller reduction when supported.
`MBS_GPU_NO_VERTEX_CACHE`	Disable cached/packed vertex path for debugging.
`MBS_GPU_TIMING`	Print CUDA timing breakdowns.

Variable	Meaning
`MBS_GPU_BLOCK`	Override CUDA kernel block size.
`MBS_HOST_MEM_FRACTION`	Host-memory fraction for oldauto/random chunking.
`MBS_HOST_MEM_RESERVE_MB`	Host memory reserve in MB.
`MBS_HOST_MEM_BUDGET_MB`	Hard host memory budget, also set for parallel children.
`MBS_FFT_PHI_FACTOR`	Legacy FFT compression-factor override when `--fft-factor` is absent; the explicit CLI value has priority.
`MBS_GPU_MULTI=0`	Disable automatic multi-orientation GPU batching.
`MBS_GPU_MULTI_MAX=N`	Cap automatic GPU multi batching.
`MBS_GPU_GROUPS=1`	Distribute variable latitude-phi theta groups across visible GPUs.
`MBS_GPU_MULTI_K_FULL=1`	Experimental fused multi-`k_eq` diffraction.
`MBS_BEAM_LOG=PATH`	Write the first handled PO beam set to the explicit diagnostic path. No beam log is written by default.
`MBS_SHARED_BETA_GROUP=N`	Override shared-batch beta grouping.
`MBS_SHARED_ORIENT_CHUNK=N`	Override shared-batch orientation chunk size.
`MBS_PARALLEL_MEM_FRACTION`	Memory fraction used by parallel multi-size scheduler.
`MBS_PARALLEL_SHARED_KEQ=1`	Environment equivalent of shared `k_eq` batching.
MBS_PARALLEL_KEQ_BATCH_RATIO=R	Environment default for shared batch ratio.

Diagnostic variables exist for FFT refinement, autofull heuristics, trace prefilters, and internal debug ranges. They are intentionally not recommended for normal production runs; inspect `grep -R "getenv(\"MBS_" src` when debugging a specific subsystem.

11 Output

Main output is a whitespace-separated `.dat` file:

```
ScAngle 2pi*dcos M11 M12 M13 M14 M21 M22 M23 M24 M31 M32 M33 M34 M41 M42 M43 M44
```

Column	Meaning
`ScAngle`	Scattering angle theta in degrees.
`2pi*dcos`	Solid-angle ring weight for this theta row.
`Mij`	Mueller matrix elements after beam and orientation averaging.

Common companion outputs:

Output	Created by	Meaning
`_noshadow` file	`--noshadow_output`	Mueller matrix without shadow/external beam contribution.
`_betas/` directory	`--save_betas`	Per-beta Mueller diagnostics.
checkpoint files	`--checkpoint`	Resume data for long orientation-grid runs.
Jones output	`--jones`	Raw Jones matrices in supported PO modes.
particle_for_check.dat	every calculation	Effective resized particle, stored inside the result directory.
`FILE` from `--save-geometry`	explicit request	Effective scaled native particle at a stable user-selected path.
`<result>_integrals.tsv`	PO and GO output	Machine-readable primary extinction, scattering, absorption, efficiencies, and the numerical C_{sca} error estimate.

The log and TSV expose exactly one method-specific definition of each physical quantity:

$$C_{\text{ext}}, \quad C_{\text{sca}}, \quad C_{\text{abs}}, \qquad Q_{\text{ext}}, \quad Q_{\text{sca}}, \quad Q_{\text{abs}}, \qquad Q_x = \frac{C_x}{A_{\text{proj}}}.$$

For PO, C_{ext} comes from the forward-amplitude optical theorem. For an absorbing material, $C_{\text{sca}} = \sum_j M_{11}(\theta_j)\Delta\Omega_j$ and $C_{\text{abs}} = C_{\text{ext}} - C_{\text{sca}}$. For a nonabsorbing material, conservation gives $C_{\text{sca}} =$

C_{ext} and $C_{\text{abs}} = 0$. For GO, $C_{\text{ext}} = A_{\text{proj}}$; the absorbing case uses the total outgoing beam energy for C_{sca} and the difference for C_{abs}. The nonabsorbing GO case again enforces $C_{\text{sca}} = C_{\text{ext}}$ and $C_{\text{abs}} = 0$. Legacy PO/GO hybrids and duplicate labels are deliberately omitted from the report.

The C_{sca} integration estimate compares the full theta grid with a nested grid formed from every second theta row:

$$\hat{\epsilon}_{\text{sca}} = \frac{|C_{\text{sca},h} - C_{\text{sca},2h}|}{\max(|C_{\text{sca},h}|, |C_{\text{sca},2h}|)}.$$

For a nonabsorbing run the report conservatively takes the larger of this estimate and the independent conservation mismatch: the M_{11} integral versus optical-theorem extinction in PO, or outgoing beam sum versus projected area in GO. Fewer than five theta rows cannot form the estimate and are marked `unavailable`. A value above 1 percent produces `check_csca_integration`. This is an a-posteriori resolution check, not a rigorous error bound and not an orientation-convergence estimate.

The TSV columns are `method`, `label`, `status`, the six C/Q values, and `Csca_error_estimate_rel`. It contains no legacy, GO-in-PO, or PO-in-GO alternatives.

The startup and final log blocks also record hostname, CPU model, logical and physical core counts, effective OpenMP thread count, total/available RAM, process RSS and peak RSS, visible GPU models, and active-device VRAM when CUDA is available. The final log contains the FFT and cutoff audit blocks described above. These records make timing and accuracy comparisons attributable to a specific resource state; GPU timings obtained while another process occupies the device are not comparable to an idle-device benchmark.

Normal runs do not create diagnostic files in the current working directory. Use `MBS_BEAM_LOG=PATH` only when an explicit first-beam dump is needed.

For nonabsorbing runs physical absorption is fixed to zero. The reported C_{sca} error remains a convergence diagnostic and does not replace inspection of the output Mueller matrix.